

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение высшего
образования
«Костромской государственный университет»
(КГУ)

Институт _Высшая ИТ-школа_____

Кафедра _Информационные системы и технологии_____

Направление подготовки _ 09.03.02 «Информационные системы и
технологии»_____

Направленность _«Информационные технологии в
бизнесе»_____

**РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ MINDVAULT ДЛЯ ХРАНЕНИЯ И
ИНТЕЛЛЕКТУАЛЬНОЙ ОБРАБОТКИ ПОЛЬЗОВАТЕЛЬСКИХ ДАННЫХ**
Выпускная квалификационная работа

Исполнитель _____ Рачинский Иван Александрович,
гр. 22-ИСбо-4
(подпись) (Ф.И.О., группа)

Руководитель _____ Мозохин Александр Евгеньевич
(подпись) (Ф.И.О.)

Кострома
2026 г.

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение высшего
образования

«Костромской государственный университет»

(КГУ)

Институт Высшая ИТ-школа

Кафедра Информационные системы и технологии

УТВЕРЖДАЮ

Зав. кафедрой _____

« ____ » _____ 2026 г.

ЗАДАНИЕ

на выполнение выпускной квалификационной работы

Студенту Рачинский Иван Александрович

Направление подготовки 09.03.02 «Информационные системы и технологии»

Тема ВКР «Разработка веб-приложения MindVault для хранения и интеллектуальной обработки пользовательских данных»

(Утверждена приказом по университету от « ____ » _____ 2026 г. № _____)

Срок сдачи студентом законченной ВКР _____

Исходные данные к ВКР: исходный код веб-приложения MindVault, требования к проекту, материалы по проектированию информационных систем, документация используемых технологий.

Таблица 1 - Состав разделов выпускной квалификационной работы

Наименование раздела ВКР	Перечень графического материала и результатов	Консультанты
Теоретическая часть	Анализ предметной области; обзор аналогов; формулировка требований; выбор инструментов разработки; описание актуальности задачи.	
Проектирование функционала системы	Диаграмма развертывания; модель данных; схема обработки пользовательского сообщения; схема обработки файла; пользовательские сценарии; структура интерфейса.	

Реализация системы	Описание клиентской и серверной частей; работа с БД; загрузка файлов; логика папок; ИИ-ассистент; Docker-развертывание; тестирование подсистем.	
--------------------	---	--

Дата выдачи задания «____» _____ 2026 г.

Руководитель _____ (Мозохин Александр Евгеньевич)

Задание принял к исполнению _____ (Рачинский Иван Александрович)

АННОТАЦИЯ

Объектом исследования является процесс хранения, систематизации и последующей обработки пользовательской цифровой информации в персональных информационных системах.

Предметом исследования являются архитектурные и программные решения, позволяющие объединить файловое хранилище, заметки, списки, напоминания и интеллектуальный чатовый интерфейс в одном веб-приложении.

Цель работы состоит в разработке веб-приложения MindVault для хранения и интеллектуальной обработки пользовательских данных. Для достижения цели были выполнены анализ предметной области, обзор аналогов, формализация требований, проектирование архитектуры, реализация клиентской и серверной частей, модели данных, подсистемы файлов, папок, напоминаний и ИИ-ассистента.

Методами исследования являются сравнительный анализ аналогов, структурное проектирование информационной системы, моделирование пользовательских сценариев, проектирование базы данных, разработка REST API, функциональное тестирование, unit-тестирование бизнес-логики и ручная проверка пользовательского интерфейса.

Практическим результатом является рабочий прототип MindVault. Система позволяет пользователю сохранять материалы через чатовый интерфейс или специализированные разделы, загружать файлы, создавать заметки, списки и напоминания, распределять данные по папкам и обращаться к ассистенту за пояснениями по сохраненным материалам.

Новизна работы заключается не в создании еще одного файлового хранилища, а в объединении привычных сценариев личного цифрового архива с контролируемым ИИ-слоем. Ассистент не заменяет серверную логику, а работает

поверх нее: реальные действия подтверждаются backend-операциями и только затем отражаются в интерфейсе.

Ключевые слова: ВЕБ-ПРИЛОЖЕНИЕ, ПЕРСОНАЛЬНОЕ ХРАНИЛИЩЕ, ИСКУССТВЕННЫЙ ИНТЕЛЛЕКТ, ЗАМЕТКИ, ФАЙЛЫ, НАПОМИНАНИЯ, ПАПКИ, REACT, VITE, TYPESCRIPT, EXPRESS, POSTGRESQL, DRIZZLE ORM, API, UX/UI.

РЕФЕРАТ

Тема: «Разработка веб-приложения MindVault для хранения и интеллектуальной обработки пользовательских данных».

Исполнитель: Рачинский Иван Александрович.

Руководитель: Мозохин Александр Евгеньевич.

Работа содержит 71 страницу машинописного текста, 8 рисунков, 26 таблиц, список литературы из 27 источников.

Ключевые слова: персональное хранилище, веб-приложение, искусственный интеллект, заметки, файлы, списки, напоминания, папки, чатовый интерфейс, база данных, API, адаптивный интерфейс.

Цель выпускной квалификационной работы - разработать веб-приложение MindVault, позволяющее хранить пользовательские материалы и работать с ними через единый чатовый интерфейс с поддержкой интеллектуального ассистента.

В ходе работы были решены задачи анализа предметной области и аналогов, определения функциональных и нефункциональных требований, выбора технологического стека, проектирования архитектуры приложения, реализации frontend и backend, разработки модели данных, подсистемы файлов, папок, заметок, списков и напоминаний, а также тестирования ключевых сценариев.

Техническая реализация выполнена с использованием React, Vite, TypeScript, Tailwind CSS и TanStack Query на клиентской стороне, Node.js и Express на серверной стороне, PostgreSQL и Drizzle ORM для хранения данных. Для валидации запросов используется Zod, для запуска в воспроизводимой среде - Docker Compose. Для извлечения текста из поддерживаемых файлов используются pdf-parse и mammoth.

Практическая значимость результата состоит в создании основы персонального цифрового пространства, которое снижает разрозненность пользовательских данных. В отличие от обычного файлового архива, MindVault

хранит материалы в единой модели и позволяет обращаться к ним через ассистента.

В перспективе проект может быть расширен семантическим поиском, векторной базой данных, OCR для изображений и сканированных PDF, полноценной календарной интеграцией, совместной работой пользователей и production-развертыванием с доменом и HTTPS.

ОГЛАВЛЕНИЕ

АННОТАЦИЯ	4
РЕФЕРАТ	6
ОГЛАВЛЕНИЕ	8
ПЕРЕЧЕНЬ УСЛОВНЫХ СОКРАЩЕНИЙ	10
ВВЕДЕНИЕ	11
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	14
1.1 Проблема хранения пользовательской информации	14
1.2 Обзор аналогов и существующих способов решения задачи	15
1.3 Применение ИИ в персональных информационных системах	17
1.4 Требования, вытекающие из анализа предметной области	18
1.5 Выводы по главе	19
2 ПОСТАНОВКА ЗАДАЧИ И ТРЕБОВАНИЯ К СИСТЕМЕ	20
2.1 Цель и задачи разработки	20
2.2 Функциональные требования	20
2.3 Нефункциональные требования	21
2.4 Пользовательские сценарии	22
2.5 Требования к обработке пользовательского ввода	24
2.6 Выбор инструментария	25
2.7 Выводы по главе	27
3 ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ MINDVAULT	28
3.1 Общая архитектура системы	28
3.2 Клиентская часть	29
3.3 Серверная часть и API	30
3.4 Проектирование базы данных	31
3.5 Логика обработки сообщений	33
3.6 Логика работы с файлами	34
3.7 Проектирование интерфейса	36
3.8 Выводы по главе	37
4 ТЕХНОЛОГИЧЕСКАЯ РЕАЛИЗАЦИЯ	38
4.1 Организация проекта	38
4.2 Реализация клиентской части	38
4.3 Реализация серверной части	39
4.4 Реализация базы данных	40
4.5 Реализация загрузки и обработки файлов	41
4.6 Реализация заметок, списков и напоминаний	41
4.7 Реализация ИИ-ассистента	42
4.8 Адаптивность и доработка интерфейса	43
4.9 Обработка ошибок и устойчивость	44
4.10 Выводы по главе	45
5 ТЕСТИРОВАНИЕ И АНАЛИЗ РАБОТЫ СИСТЕМЫ	46
5.1 Методика тестирования	46
5.2 Функциональное тестирование	46
5.3 Тестирование нестандартных пользовательских вводов	47
5.4 Тестирование работы с файлами	48
5.5 Тестирование адаптивности интерфейса	49
5.6 Анализ эффективности решения	50

5.7 Ограничения текущей версии	51
5.8 Выводы по главе	52
ЗАКЛЮЧЕНИЕ	53
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	55

ПЕРЕЧЕНЬ УСЛОВНЫХ СОКРАЩЕНИЙ

Таблица 2 - Перечень условных сокращений

Сокращение	Расшифровка
API	Application Programming Interface - программный интерфейс приложения.
CRUD	Create, Read, Update, Delete - базовые операции создания, чтения, изменения и удаления данных.
CSS	Cascading Style Sheets - язык описания внешнего вида веб-страницы.
DTO	Data Transfer Object - объект передачи данных между слоями приложения.
HTTP	HyperText Transfer Protocol - протокол передачи данных в сети.
HTTPS	Защищенная версия HTTP с шифрованием соединения.
JSON	JavaScript Object Notation - формат обмена структурированными данными.
LLM	Large Language Model - большая языковая модель.
MVP	Minimum Viable Product - минимально жизнеспособная версия продукта.
ORM	Object-Relational Mapping - объектно-реляционное отображение.
SPA	Single Page Application - одностраничное веб-приложение.
UI	User Interface - пользовательский интерфейс.
UX	User Experience - пользовательский опыт.
БД	База данных.
ВКР	Выпускная квалификационная работа.
ИИ	Искусственный интеллект.
ИС	Информационная система.

ВВЕДЕНИЕ

В повседневной работе пользователь сталкивается не с одной крупной базой данных, а с множеством небольших цифровых фрагментов: учебными документами, ссылками, скриншотами, заметками, списками покупок, идеями для проекта, напоминаниями о встречах и сообщениями, которые нужно не потерять. Каждый такой фрагмент сам по себе невелик, но вместе они образуют личный информационный архив. Проблема заключается в том, что этот архив редко находится в одном месте.

На практике часть материалов хранится в облачном диске, часть - в мессенджере, часть - в заметочнике, а задачи и сроки переносятся в отдельный календарь или task-менеджер. Такой подход кажется привычным, но он создает постоянные переключения между сервисами. Пользователь запоминает смысл информации, но не всегда помнит, где именно она была сохранена. В результате время тратится не на работу с материалом, а на его поиск.

Ситуация стала особенно заметной на фоне развития ИИ-ассистентов. Языковые модели уже способны объяснять текст, кратко пересказывать документы и помогать с формулировками. Однако в обычном режиме они не связаны с реальным личным архивом пользователя. Ассистент может ответить на вопрос, но не всегда может надежно создать заметку, сохранить файл, найти ранее загруженный материал или поставить напоминание в конкретной системе.

В рамках данной работы рассматривается разработка MindVault - веб-приложения, которое объединяет несколько привычных сценариев: личное файловое хранилище, заметочник, списки, напоминания, папки и чат с ассистентом. Центральной идеей является не простое добавление ИИ в интерфейс, а построение системы, где ассистент работает поверх реальных данных и реальных backend-операций.

Актуальность темы определяется двумя факторами. Во-первых, растет объем личной цифровой информации, которую нужно хранить и быстро находить. Во-вторых, пользователи все чаще ожидают от приложений естественно-языкового интерфейса. Поэтому перспективным является подход, при котором пользователь может не выбирать заранее сложную форму, а отправить сообщение, файл или команду в один интерфейс, после чего система поможет структурировать данные.

Целью выпускной квалификационной работы является разработка веб-приложения MindVault для хранения и интеллектуальной обработки пользовательских данных.

Для достижения поставленной цели были решены следующие задачи: проведен анализ предметной области и существующих решений; сформулированы функциональные и нефункциональные требования; определены пользовательские сценарии; выбран технологический стек; спроектирована архитектура приложения; разработана модель данных; реализованы клиентская и серверная части; реализованы подсистемы файлов, заметок, списков, напоминаний, папок и ассистента; проведено тестирование основных и нестандартных сценариев.

Объектом исследования является процесс хранения и обработки личной цифровой информации пользователя. Предметом исследования являются методы проектирования и реализации веб-приложения, объединяющего хранилище данных, чатовый интерфейс и интеллектуальную обработку запросов.

Практическая значимость работы состоит в создании прототипа персонального цифрового пространства. Разработанная система может использоваться как основа для дальнейшего развития продукта: подключения семантического поиска, векторной базы данных, OCR, календарной интеграции и совместной работы пользователей.

Выпускная квалификационная работа состоит из введения, пяти глав, заключения и списка использованных источников. В первой главе

рассматривается предметная область и аналоги. Во второй главе формулируются требования. В третьей главе описывается проектирование системы. В четвертой главе приведена технологическая реализация. В пятой главе представлены результаты тестирования и анализ ограничений MVP.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Проблема хранения пользовательской информации

Организация личной информации давно перестала быть задачей только файловой системы. Пользователь работает не только с документами, но и с короткими мыслями, сообщениями, ссылками, изображениями, учебными материалами и временными задачами. Для разных типов данных исторически появились разные приложения, поэтому личная информация постепенно разделилась между несколькими рабочими пространствами.

Наиболее простой подход - хранение файлов в папках. Он понятен, но требует от пользователя постоянной дисциплины: нужно выбрать папку, правильно назвать файл, не забыть перенести его после загрузки и поддерживать порядок. В учебной и проектной работе такая схема быстро усложняется, потому что один файл может относиться сразу к нескольким контекстам: дисциплине, задаче, сроку, преподавателю или этапу проекта.

Заметочки решают другую часть задачи. Они позволяют быстро записывать идеи, но слабее работают с файлами и сроками. Task-менеджеры, наоборот, хорошо работают с дедлайнами, однако не подходят для хранения длинных материалов. Мессенджеры удобны для быстрого сброса информации, но не дают полноценной структуры. В итоге пользователь вынужден сам быть связующим звеном между сервисами.

Отдельно следует отметить разницу между хранением и использованием информации. Файл может быть сохранен, но это еще не означает, что с ним удобно работать. Пользователь часто хочет задать вопрос по содержимому документа, попросить краткое резюме, извлечь список задач или найти похожую заметку. Обычное хранилище такую работу не выполняет, а отдельный ИИ-чат не имеет доступа к данным пользователя без дополнительной интеграции.

Именно этот разрыв стал исходной проблемой проекта MindVault. Требовалось создать систему, в которой данные не просто лежат в папках, а становятся частью единого пользовательского контекста. При этом система не должна быть сложной базой знаний с большим количеством правил. Основной сценарий должен оставаться простым: пользователь открывает приложение, пишет в чат или загружает файл, а система помогает сохранить и структурировать материал.

Для учебного проекта эта задача имеет дополнительную ценность. Она позволяет показать не только верстку или CRUD-интерфейс, но и полный цикл проектирования информационной системы: анализ аналогов, постановку требований, проектирование модели данных, реализацию API, интеграцию внешнего сервиса и тестирование нестандартных вводов.

Проблема усложняется тем, что пользовательская информация редко бывает строго одного типа. Например, файл с лекцией может одновременно быть учебным материалом, источником для заметки и основанием для напоминания о подготовке к зачету. Если система хранит эти элементы отдельно, пользователю приходится вручную восстанавливать связи между ними. Для MindVault эта связь должна быть частью продукта, а не внешней привычкой пользователя.

1.2 Обзор аналогов и существующих способов решения задачи

Для понимания требований к MindVault были рассмотрены решения, которыми пользователи обычно заменяют персональное хранилище: облачные диски, заметочники, базы знаний, task-менеджеры, мессенджеры и чат-ассистенты. Они не являются прямыми конкурентами в узком смысле, но закрывают отдельные части той же пользовательской потребности.

Таблица 3 - Сравнение существующих решений

Решение	Лицензия и доступ	Полезные функции	Ограничения для задачи MindVault
Google Drive	Бесплатная версия и платные тарифы, закрытый код	Хранение файлов, синхронизация, доступ по ссылке, базовый поиск	Фокус на файлах; заметки, напоминания и диалоговая работа с

			содержимым не являются центральным сценарием.
Notion	Бесплатная версия и платные тарифы, закрытый код	Страницы, базы данных, шаблоны, совместная работа	Высокий порог настройки; пользователь сам проектирует структуру и поддерживает порядок.
Telegram Saved Messages	Бесплатный сервис, закрытый код	Быстрый сброс ссылок, сообщений и файлов в один поток	Слабая структуризация, нет надежной модели папок и действий над данными приложения.
Obsidian	Бесплатно для личного использования, закрытый код с локальными файлами Markdown	Локальная база знаний, связи между заметками, плагины	Хорош для заметок, но не является веб-хранилищем файлов и напоминаний из коробки.
Evernote	Бесплатная версия и платные тарифы, закрытый код	Заметки, вложения, поиск, сканирование документов	Ограниченная командная логика и зависимость от ручной организации материалов.
Todoist / Google Tasks	Бесплатные и платные варианты, закрытый код	Задачи, дедлайны, напоминания, списки	Не решают задачу хранения файлов и интеллектуального анализа документов.
ChatGPT / Gemini	Бесплатные и платные тарифы, закрытые модели	Ответы на вопросы, анализ текста, генерация структуры	Без интеграции с БД не выполняют надежные операции над пользовательским хранилищем.

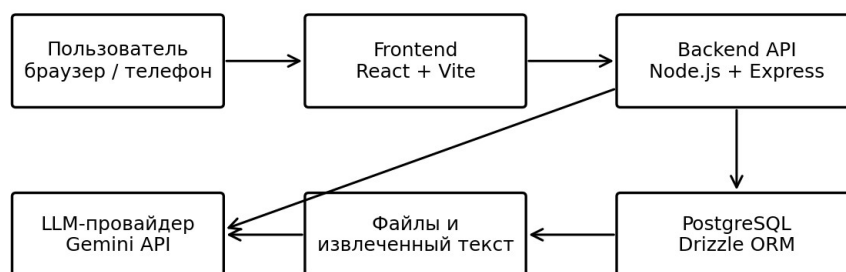
Анализ показывает, что большинство инструментов эффективны в своей области, но не дают единого сценария. Google Drive удобен для долговременного хранения файлов, однако не превращает документы в активный контекст ассистента. Notion силен как база знаний, но требует ручного проектирования пространства. Telegram удобен скоростью, но его сохраненные сообщения со временем превращаются в длинную ленту без нормальной типизации. Task-менеджеры закрывают сроки, но не связаны с файлами и заметками.

MindVault проектируется как компромисс между простотой мессенджера и структурированностью специализированных приложений. Пользователь может начать с одного чата, но сохраненная информация не остается просто сообщением: она становится объектом с типом, папкой, датой, метаданными и возможностью дальнейшей обработки.

Сравнение аналогов показывает, что разрабатываемое приложение не должно пытаться полностью заменить каждую специализированную систему.

Например, Google Drive лучше подходит для корпоративной совместной работы с большим числом файлов, а Notion - для сложных рабочих баз. MindVault ориентирован на другой сценарий: личное быстрое сохранение и последующую работу с материалом через единый ассистентский интерфейс.

С точки зрения ВКР важно не просто перечислить известные сервисы, а показать, почему разработка собственного приложения оправдана. У MindVault есть прикладное отличие: он объединяет чат, реальные операции с данными и типизированное хранение объектов. Поэтому проект не сводится к копированию существующего облачного диска или заметочника.



Данные пользователя не обрабатываются только моделью: действия выполняются через backend и БД

Рисунок 1 - Общая архитектурная идея MindVault как единого интерфейса к данным

1.3 Применение ИИ в персональных информационных системах

Искусственный интеллект в персональных информационных системах может выполнять несколько функций: классифицировать пользовательские сообщения, извлекать смысл из документов, составлять краткое содержание, помогать с поиском и преобразовывать естественный язык в структурированные действия. Для MindVault наиболее важны не только ответы модели, но и связь ответа с данными приложения.

Главный риск при интеграции LLM заключается в том, что языковая модель склонна формулировать уверенный текст даже в ситуациях, когда реальное действие не было выполнено. Для обычного чат-бота это выглядит как неточность, а для информационной системы - как функциональная ошибка. Если ассистент сообщил «заметка создана», запись должна реально появиться в базе данных.

Поэтому в MindVault выбран подход, при котором ассистент не является единственным исполнителем команд. Система сначала пытается определить, является ли сообщение явной командой. Если команда однозначна, действие выполняется на backend. Если сообщение спорное, пользователю предлагаются кнопки подтверждения. Если сообщение является обычным вопросом, оно остается сообщением чата и не создает лишних объектов.

Такое разделение снижает риск «мусорного» накопления данных. Например, фраза «как лучше подготовиться к защите диплома» не должна автоматически становиться заметкой или напоминанием. И наоборот, сообщение «напоминание завтра в 10:00 отправить презентацию» должно создать объект типа reminder, потому что пользователь явно указал команду, дату, время и содержание.

Важной особенностью является ограничение контекста. Передавать в LLM всю базу пользователя нецелесообразно и небезопасно. Backend собирает релевантные данные: последние сообщения, подходящие заметки, имена файлов, извлеченный текст и источники. Такой подход позволяет ассистенту отвечать по материалам пользователя, не превращая каждый запрос в полную выгрузку хранилища.

Для пользователя такая схема выглядит простой: он видит обычный чат. Но внутри приложения чат является не только окном переписки, а интерфейсом к данным. Это требует аккуратного проектирования, потому что ошибка в классификации сообщения может привести к потере доверия к системе.

1.4 Требования, вытекающие из анализа предметной области

Таблица 4 - Выводы из анализа аналогов

Проблема	Как проявляется у пользователя	Требование к MindVault
Разрозненность данных	Файлы, заметки и задачи лежат в разных сервисах	Единая модель пользовательских материалов.
Сложность ручной сортировки	Пользователь откладывает организацию и складывает все в одну папку	Быстрое сохранение через чат и последующее распределение по папкам.
Недоверие к действиям ассистента	Ассистент может написать «готово», хотя объект не создан	Подтверждать действие только после успешной backend-операции.
Неполная обработка файлов	Документы сохранены, но их содержимое нельзя обсудить	Извлекать текст из поддерживаемых форматов и честно показывать ограничения.
Мобильное использование	На телефоне интерфейс часто ломается или перекрывает ввод	Адаптивная навигация, drawer, корректный composer.

На основе анализа была определена «изюминка» проекта: MindVault должен не просто хранить разные типы объектов, а связывать их с ассистентом и при этом оставлять управление данными на стороне приложения. Это отличает систему от простого чата с ИИ и от обычного облачного диска.

Также из анализа следует, что приложение не должно строиться только вокруг автоматической сортировки. Полностью автоматическая классификация на раннем этапе может ошибаться. Поэтому в MVP сочетались два подхода: ручная структура папок и подсказки ассистента. Такой вариант устойчивее для демонстрации и понятнее для пользователя.

1.5 Выводы по главе

В первой главе была рассмотрена предметная область персонального хранения информации. Анализ показал, что существующие инструменты решают отдельные задачи, но не закрывают целиком сценарий «быстро сохранить - структурировать - обсудить с ассистентом - вернуться к материалу позже». Поэтому разработка MindVault является актуальной прикладной задачей. Полученные выводы использованы при формулировке требований к системе во второй главе.

2 ПОСТАНОВКА ЗАДАЧИ И ТРЕБОВАНИЯ К СИСТЕМЕ

2.1 Цель и задачи разработки

Целью разработки MindVault является создание веб-приложения, которое позволяет пользователю хранить личные цифровые материалы в одном месте и работать с ними через понятный интерфейс, близкий к обычному чату. При постановке задачи учитывалось, что пользователь не всегда хочет заранее выбирать тип сущности, папку или форму ввода. В реальном использовании чаще возникает простая потребность: быстро отправить мысль, загрузить документ, записать задачу, а затем при необходимости найти этот материал и обработать его.

Поэтому задача разработки была сформулирована не как создание еще одного файлового менеджера или заметочника, а как создание небольшого персонального информационного пространства. В нем файл, заметка, список и напоминание рассматриваются как разные представления пользовательского материала, а чатовый интерфейс используется как общий вход в систему.

Для достижения поставленной цели были определены следующие задачи: выполнить анализ аналогов; выделить ограничения существующих решений; определить обязательные и желательные требования; спроектировать архитектуру веб-приложения; разработать модель данных; реализовать клиентскую и серверную части; реализовать загрузку и хранение файлов; реализовать механизм папок; разработать правила обработки сообщений ассистентом; проверить систему на основных и нестандартных сценариях.

2.2 Функциональные требования

Функциональные требования были сформированы на основании анализа аналогов и практических сценариев, которые возникали во время разработки. Для первой версии важно было не охватить все возможные функции персонального рабочего пространства, а реализовать связанный набор возможностей, который

можно продемонстрировать и развивать дальше. Поэтому требования были разделены на обязательные, желательные и перспективные.

К обязательным требованиям отнесены функции, без которых приложение не решает свою основную задачу: чат, хранение материалов, загрузка файлов, разделы заметок и напоминаний, работа с папками и базовая обработка пользовательских сообщений. Желательные требования повышают удобство системы, но могут развиваться постепенно: предпросмотр файлов, улучшенная фильтрация, управление действиями ассистента через кнопки, расширенная обработка нестандартных запросов.

Таблица 5 - Функциональные требования к системе

№	Требование	Приоритет и комментарий
1	Единый чатовый интерфейс	Обязательное. Чат является главным способом взаимодействия пользователя с системой.
2	Создание заметок	Обязательное. Пользователь может сохранить короткий текст как отдельный материал.
3	Создание списков	Обязательное. Списки нужны для задач, покупок, учебных материалов и личных планов.
4	Создание напоминаний	Обязательное. Система распознает простые временные выражения и сохраняет напоминание.
5	Загрузка файлов	Обязательное. Файл должен сохраняться, отображаться в интерфейсе и быть связан с пользователем.
6	Сортировка по папкам	Обязательное. Папки помогают не превращать поток сообщений в неуправляемую ленту.
7	Интеллектуальный ассистент	Обязательное. Ассистент помогает отвечать на вопросы и работать с сохраненными данными.
8	Предпросмотр файлов	Желательное. В первой версии реализуется ограниченно и зависит от типа файла.
9	Кнопки уточнения действия	Желательное. Полезны при спорных командах и ошибочной классификации запроса.
10	Семантический поиск	Перспективное. Требуется отдельного индекса и более сложной инфраструктуры.

2.3 Нефункциональные требования

Нефункциональные требования оказали существенное влияние на архитектуру системы. MindVault должен быть не только функциональным, но и удобным для демонстрации, сопровождения и дальнейшей доработки. В условиях выпускной квалификационной работы особенно важны простота развертывания, понятность структуры проекта и возможность показать работающий результат на защите.

Ключевым нефункциональным требованием стала предсказуемость поведения. Пользователь должен понимать, что произойдет после ввода сообщения. Если ассистент создает заметку или напоминание, это действие должно действительно выполняться в базе данных. Если запрос является обычным вопросом, система не должна превращать его в объект хранения без явного намерения пользователя.

Таблица 6 - Нефункциональные требования к системе

№	Требование	Приоритет и влияние на проектирование
1	Простота интерфейса	Высокий. Сокращает число действий при сохранении и поиске материалов.
2	Адаптивность	Высокий. Интерфейс должен корректно работать на desktop и mobile.
3	Надежность операций	Высокий. Ответ о создании объекта формируется только после успешной записи в БД.
4	Расширяемость	Средний. Единая модель материалов упрощает добавление новых типов данных.
5	Безопасность хранения	Средний. В учебном прототипе учитывается разграничение пользовательских данных.
6	Простота развертывания	Средний. Docker Compose облегчает запуск проекта на другом компьютере или сервере.
7	Низкий порог входа	Высокий. Пользователь может начать работу без предварительной настройки пространства.

2.4 Пользовательские сценарии

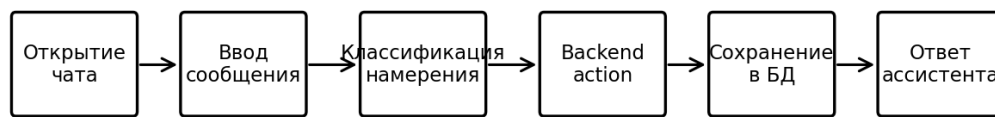
Основные сценарии были описаны в формате user story. Такой способ удобен тем, что показывает не только функцию, но и причину ее появления. Для MindVault это важно: многие функции выглядят простыми по отдельности, однако ценность приложения возникает именно при их совместной работе.

Например, пользователь может загрузить файл с учебными материалами, попросить ассистента объяснить его содержимое, сохранить краткий вывод как заметку и поставить напоминание вернуться к нему позже. В отдельных сервисах такой сценарий потребовал бы переключения между диском, чат-ботом, заметочником и календарем. В MindVault он должен выполняться в одном интерфейсе.

Таблица 7 - Основные пользовательские сценарии

№	Роль	Сценарий	Ожидаемый результат
1	Пользователь	Отправить заметку в чат	Текст сохраняется как заметка и появляется в соответствующем разделе.
2	Пользователь	Загрузить файл	Файл сохраняется, отображается в разделе файлов и доступен для дальнейшей работы.
3	Пользователь	Создать список через сообщение	Список разбивается на пункты и сохраняется в базе данных.
4	Пользователь	Написать 'напомни завтра'	Создается напоминание с распознанной датой.
5	Пользователь	Задать вопрос ассистенту	Система отвечает без создания

№	Роль	Сценарий	Ожидаемый результат
			лишнего объекта хранения.
6	Пользователь	Разложить материалы по папкам	Объекты фильтруются и отображаются внутри выбранной папки.
7	Пользователь	Открыть приложение с телефона	Навигация и поле ввода остаются доступными на малом экране.



Спорные случаи не меняют данные сразу - пользователю предлагается подтвердить действие

Рисунок 2 - Событийная модель основных сценариев MindVault

2.5 Требования к обработке пользовательского ввода

Отдельным требованием стала корректная обработка свободного ввода. В отличие от классической формы, чатовый интерфейс не ограничивает пользователя заранее заданными полями. Это делает систему удобнее, но одновременно увеличивает число неоднозначных ситуаций. Например, сообщение «заметка список напоминание» содержит сразу несколько ключевых слов, но не содержит полезного содержания и не должно автоматически создавать три объекта.

Для решения этой проблемы была выбрана комбинированная схема. Явные команды распознаются простыми правилами, а не через генеративную модель.

Такой подход проще тестировать и легче объяснить на защите. Если сообщение начинается с ключевого слова «заметка», «список» или «напоминание», система пытается выполнить соответствующее действие. Если намерение неочевидно, приложение либо отвечает как ассистент, либо предлагает пользователю уточнить действие.

При проектировании этого механизма учитывалось требование доверия к системе. Пользователь не должен получать сообщение «записал», если запись не появилась в базе данных. Поэтому подтверждение формируется только после успешного ответа серверной части. Ошибки API или базы данных должны отображаться как ошибки, а не маскироваться обычным ответом ассистента.

Таблица 8 - Правила обработки пользовательского ввода

Тип ввода	Пример	Решение системы
Явная заметка	заметка: идея для диплома	Создать объект типа note.
Явный список	список: купить хлеб, молоко	Создать объект типа list.
Явное напоминание	напомни завтра в 10 отправить отчет	Создать объект типа reminder.
Обычный вопрос	как лучше оформить раздел тестирования?	Ответить без сохранения нового материала.
Смешанный ввод	заметка список напоминание	Не создавать объект автоматически, запросить уточнение.
Пустое сообщение		Не отправлять запрос и не создавать запись.
Команда без содержания	заметка	Попросить пользователя ввести текст заметки.

2.6 Выбор инструментария

Выбор технологического стека выполнялся с учетом требований к скорости разработки, удобству демонстрации, наличию готовых библиотек и возможности развертывания проекта как веб-сервиса. В отличие от учебного десктопного приложения, MindVault должен открываться в браузере, корректно работать на разных устройствах и иметь разделение на клиентскую и серверную части.

Для клиентской части были выбраны React, Vite, TypeScript и Tailwind CSS. React хорошо подходит для интерфейсов с большим числом состояний, Vite ускоряет разработку и сборку, TypeScript помогает снизить число ошибок при изменении структуры данных, а Tailwind CSS позволяет быстро поддерживать единый визуальный стиль. Для серверной части использовались Node.js и Express, так как этот стек хорошо сочетается с frontend-разработкой на JavaScript/TypeScript и не требует перехода на другой язык.

Для хранения данных была выбрана PostgreSQL. В проекте используются не только простые записи, но и связанные сущности: папки, элементы, сообщения, вложения и метаданные. Поэтому реляционная модель оказалась удобнее простого хранения в JSON-файлах. Drizzle ORM применялась для описания схемы и запросов на уровне кода, а Zod - для проверки входных данных.

Таблица 9 - Сравнение технологий клиентской части

Критерий	React + Vite	Альтернативы	Вывод
Скорость разработки	Высокая	Vue/Nuxt - высокая; Angular - средняя	React + Vite подходит для быстрого прототипирования.
Работа со сложным состоянием	Удобная	Angular удобен, но тяжелее; чистый JS ограничен	Для чата и разделов удобнее компонентный подход.
Порог входа	Средний	Angular требует больше подготовки	Стек понятен и хорошо документирован.
Подходит для MVP	Да	Angular избыточен; чистый JS хуже масштабируется	Выбранный вариант оставляет запас для развития.
Причина выбора	Гибкость и развитая экосистема	Vue/Nuxt рассматривался как альтернатива	React + Vite лучше подошел к структуре проекта.

Таблица 10 - Выбранный технологический стек проекта

Компонент	Выбранное решение	Альтернативы	Обоснование
Frontend	React + Vite + TypeScript	Vue, Angular	Быстрая разработка интерактивного

Компонент	Выбранное решение	Альтернативы	Обоснование
			интерфейса и строгая типизация.
Стили	Tailwind CSS	SCSS, CSS Modules	Единый стиль без большого числа отдельных CSS-файлов.
Backend	Node.js + Express	Django, FastAPI, NestJS	Простая интеграция с TypeScript-экосистемой проекта.
База данных	PostgreSQL	SQLite, MongoDB	Подходит для связанных сущностей и надежного хранения.
ORM	Drizzle ORM	Prisma, Sequelize	Явное описание схемы и типизированные запросы.
Валидация	Zod	Joi, ручная проверка	Единая проверка входных данных на границе API.
Развертывание	Docker Compose	Ручная установка	Повторяемый запуск сервера и базы данных.

2.7 Выводы по главе

В результате постановки задачи были выделены основные требования к MindVault. Система должна объединять хранение файлов, заметки, списки, напоминания и чатовый интерфейс с ассистентом. При этом ключевой особенностью является не просто наличие ИИ, а правильное разделение между обычным ответом модели и реальными действиями приложения.

Для реализации выбран веб-стек React, Vite, TypeScript, Node.js, Express, PostgreSQL и Drizzle ORM. Такой набор технологий соответствует требованиям MVP, позволяет быстро развивать интерфейс и обеспечивает достаточно понятную архитектуру для дальнейшего сопровождения.

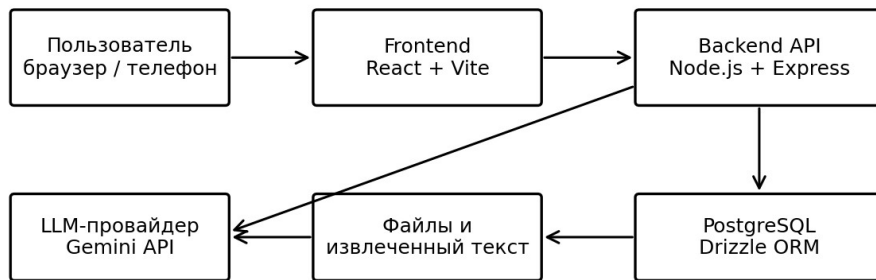
3 ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ MINDVAULT

3.1 Общая архитектура системы

MindVault проектировался как клиент-серверное веб-приложение. Такой вариант был выбран потому, что приложение должно открываться в браузере, не требовать установки на компьютер пользователя и быть пригодным для демонстрации на сервере. В отличие от локального приложения, веб-сервис проще развивать как продукт: к нему можно подключать домен, HTTPS, базу данных, файловое хранилище и внешние API.

Архитектура системы разделена на несколько крупных блоков: клиентское приложение, сервер API, база данных, файловое хранилище, модуль обработки пользовательских действий и модуль интеграции с языковой моделью. Клиент отвечает за отображение разделов и взаимодействие с пользователем. Сервер отвечает за сохранение данных, валидацию запросов, обработку файлов и выполнение действий, которые ассистент подтверждает в интерфейсе.

Такое разделение важно для надежности. Если логика создания заметок, списков и напоминаний была бы полностью перенесена на клиент, система стала бы менее защищенной и сложнее сопровождалась. Поэтому все операции изменения данных выполняются через backend. Клиент только отправляет запрос и отображает результат.



Данные пользователя не обрабатываются только моделью: действия выполняются через backend и БД

Рисунок 3 - Общая архитектура веб-приложения MindVault

Таблица 11 - Компоненты архитектуры MindVault

Компонент	Назначение	Основные обязанности
Клиентское приложение	Интерфейс пользователя	Отображение чата, файлов, заметок, списков, напоминаний и папок.
API-сервер	Обработка запросов	CRUD-операции, валидация и маршрутизация действий.
База данных	Постоянное хранение	Пользователи, папки, материалы, чаты, сообщения и метаданные.
Файловое хранилище	Хранение вложений	Сохранение загруженных файлов и информации о них.
Модуль ассистента	Интеллектуальная обработка	Ответы на вопросы, работа с контекстом и помощь при структурировании.
Парсер команд	Детерминированная обработка	Распознавание явных команд без обращения к языковой модели.

3.2 Клиентская часть

Клиентская часть построена вокруг нескольких основных экранов. Главным экраном является чат, поскольку именно через него пользователь чаще всего взаимодействует с системой. В левой части интерфейса располагается навигация

по разделам и папкам. Отдельные разделы предназначены для файлов, заметок, списков, напоминаний и настроек.

Структура интерфейса намеренно не перегружалась. Во многих сервисах проблема заключается в том, что пользователь сначала должен изучить логику продукта, а уже потом начать сохранять данные. В MindVault основной сценарий начинается сразу: пользователь пишет сообщение или прикрепляет файл. Остальные разделы нужны для просмотра и управления сохраненными объектами.

Для работы с данными на клиенте используется асинхронная загрузка через API. Такой подход позволяет не хранить все данные в состоянии интерфейса вручную и обновлять разделы после операций создания, изменения или удаления. При добавлении новой заметки или файла соответствующий раздел получает актуальные данные с сервера.

Таблица 12 - Разделы пользовательского интерфейса

Раздел	Назначение	Особенности реализации
Чат	Основное взаимодействие с ассистентом	Сообщения пользователя, ответы ассистента и прикрепленные файлы.
Файлы	Просмотр загруженных файлов	Карточки файлов, метаданные и ограниченный предпросмотр.
Заметки	Просмотр и создание текстовых записей	Фильтрация по папкам и быстрый доступ к содержимому.
Списки	Хранение структурированных пунктов	Подходит для задач, покупок и учебных материалов.
Напоминания	Контроль действий по времени	Сохраняется текст напоминания и дата выполнения.
Настройки	Параметры приложения	Служебные действия и будущие пользовательские настройки.

3.3 Серверная часть и API

Серверная часть MindVault реализует набор маршрутов, которые используются клиентским приложением. На уровне API выделяются маршруты

для элементов, папок, сообщений, загрузки файлов и взаимодействия с ассистентом. Сервер также проверяет входные данные, чтобы некорректный запрос не приводил к повреждению базы.

Важной особенностью серверной логики является разделение обычных данных и метаданных. Например, текст заметки хранится как содержимое элемента, а дополнительные сведения о файле, источниках ответа ассистента или ожидаемом действии могут сохраняться в JSON-поле `metadata`. Такой подход дает гибкость без создания большого числа отдельных таблиц для каждой мелкой детали.

API проектировался в стиле, близком к REST. Это упростило связь между клиентом и сервером: клиент обращается к понятным маршрутам, а сервер возвращает структурированный JSON. Такой формат удобен для frontend-разработки и тестирования через инструменты разработчика или отдельные HTTP-клиенты.

Таблица 13 - Основные группы API-маршрутов

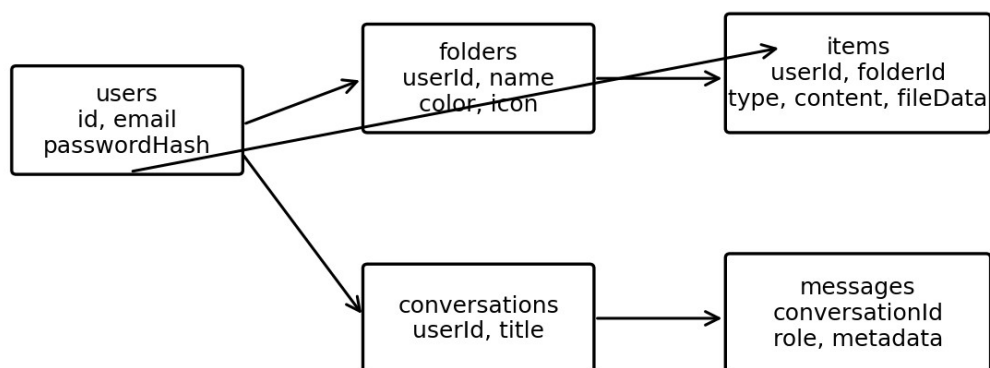
Группа маршрутов	Назначение	Комментарий
items	Создание и получение заметок, файлов, списков и напоминаний	Единая точка работы с материалами пользователя.
folders	Создание, переименование и удаление папок	Папки используются для сортировки материалов.
conversations	Работа с чатами	Позволяет хранить историю взаимодействия.
messages	Сохранение сообщений	Сообщения пользователя и ассистента связаны с конкретным чатом.
files	Загрузка и получение метаданных файлов	В первой версии загрузка связана с карточкой файла.
gemini/classify	Классификация сложного намерения	Используется как вспомогательный механизм, а не вместо backend-логики.

3.4 Проектирование базы данных

Модель базы данных строилась вокруг универсальной сущности item. В разных приложениях файлы, заметки и напоминания часто хранятся в разных моделях. Для MindVault такой подход усложнил бы интерфейс и работу ассистента. Поэтому основные пользовательские материалы объединены в одной таблице, а их различие задается полем type.

Использование единой таблицы items упрощает фильтрацию по папкам и поиск. Например, пользователь может открыть папку «Учеба» и увидеть там не только заметки, но и файлы, списки и напоминания. Для персонального хранилища это естественнее, чем жесткое разделение данных по типам.

Отдельно хранятся чаты и сообщения. Это позволяет не смешивать сами материалы пользователя с историей диалога. Ассистент может отвечать в conversation, но не каждое сообщение становится item. Такой подход напрямую решает одну из проблем, возникших в ходе разработки: система не должна сохранять каждое обращение к ассистенту как заметку или напоминание.



Единая сущность items хранит заметки, файлы, списки и напоминания через поле type

Рисунок 4 - Логическая модель базы данных MindVault

Таблица 14 - Основные таблицы базы данных

Таблица	Назначение	Ключевые поля
users	Хранение пользователей	id, email, password_hash, created_at

Таблица	Назначение	Ключевые поля
folders	Папки пользователя	id, user_id, name, parent_id, created_at
items	Основные пользовательские материалы	id, user_id, folder_id, type, title, content, metadata
conversations	Чаты пользователя	id, user_id, title, created_at
messages	Сообщения в чатах	id, conversation_id, role, content, metadata, created_at

3.5 Логика обработки сообщений

Обработка сообщений является центральной частью MindVault. Пользовательский ввод проходит несколько этапов: предварительная очистка, определение явной команды, проверка неоднозначности, выполнение backend-action или отправка запроса ассистенту. Такая цепочка делает поведение системы более управляемым.

Если пользователь вводит явную команду, например «заметка», «список» или «напоминание», система сначала пытается создать соответствующий объект. Если операция успешна, в чат добавляется ответ о сохранении. Если операция не выполнена, например из-за отсутствия текста или ошибки базы данных, пользователь получает сообщение о проблеме. Такой принцип исключает ложное подтверждение действия.

Если сообщение не является командой, оно обрабатывается как обычный запрос к ассистенту. В этом случае система может использовать релевантный контекст из базы: названия заметок, содержимое, имена файлов и папки. При этом обычный вопрос не превращается в сохраненный объект. Это важно, потому что иначе база быстро заполнялась бы мусорными записями.

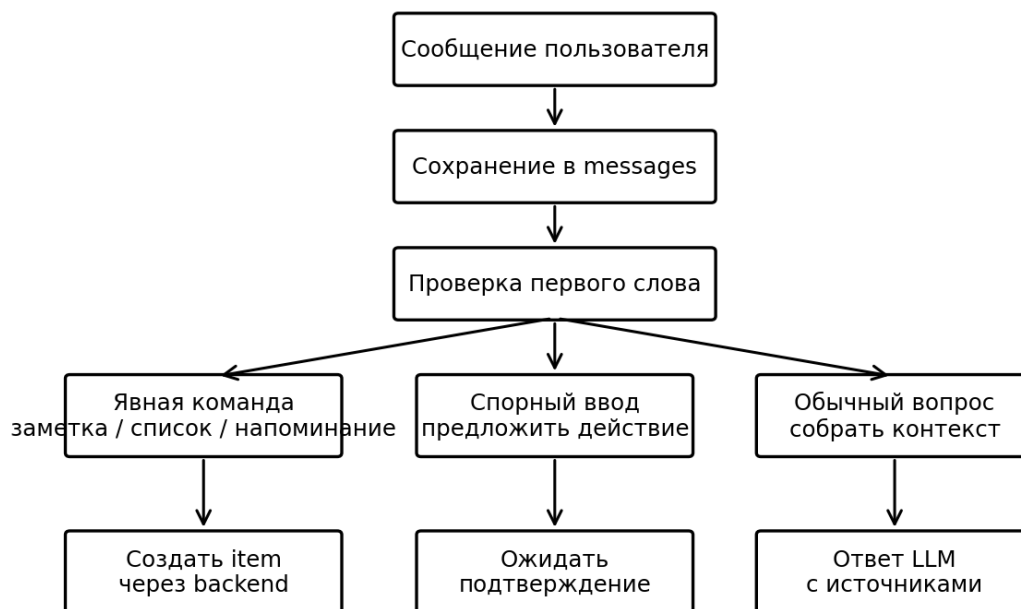


Рисунок 5 - Алгоритм обработки сообщения пользователя

Таблица 15 - Алгоритм обработки сообщения пользователя

Этап	Описание	Результат
1	Получение текста и вложений от клиента	Сформирован исходный запрос пользователя.
2	Очистка строки и проверка пустого ввода	Некорректные сообщения отсекаются до обращения к API.
3	Проверка первого слова команды	Определяется note, list, reminder или обычный вопрос.
4	Выполнение backend-action	Создается объект, если команда корректна и содержит данные.
5	Сохранение сообщения	История чата остается полной, но не каждое сообщение становится материалом.
6	Формирование ответа ассистента	Пользователь получает результат действия или просьбу уточнить запрос.

3.6 Логика работы с файлами

Файлы в MindVault являются одним из типов item. При загрузке создается запись, содержащая название, тип, размер, расширение и другие метаданные. Если файл относится к поддерживаемому текстовому формату, система пытается извлечь текстовое содержимое и сохранить его для дальнейшей работы ассистента.

В MVP поддержка разных форматов реализуется с учетом практических ограничений. Текстовые файлы можно прочитать напрямую, DOCX обрабатывается через библиотеку mammoth, PDF - через pdf-parse при наличии текстового слоя. Изображения, презентации и некоторые сложные PDF сохраняются как файлы с метаданными, но без полноценного анализа содержимого. Это честное ограничение первой версии, которое лучше, чем попытка имитировать чтение файла без реальных данных.

Такой подход позволяет сохранить файл в любом случае, даже если извлечение текста не удалось. Пользователь не теряет материал, а система может показать понятное сообщение о том, что предпросмотр или анализ ограничен для данного формата.



Если текст извлечь нельзя, файл сохраняется как metadata-only, а ассистент не выдумывает его содержимое

Рисунок 6 - Процесс загрузки и обработки файла

Таблица 16 - Поддержка форматов файлов в MVP

Тип файла	Способ обработки в MVP	Ограничения
TXT / Markdown	Прямое чтение текста	Результат зависит от кодировки файла.
PDF	Извлечение текста через pdf-parse	Сканированные PDF без OCR не читаются как текст.

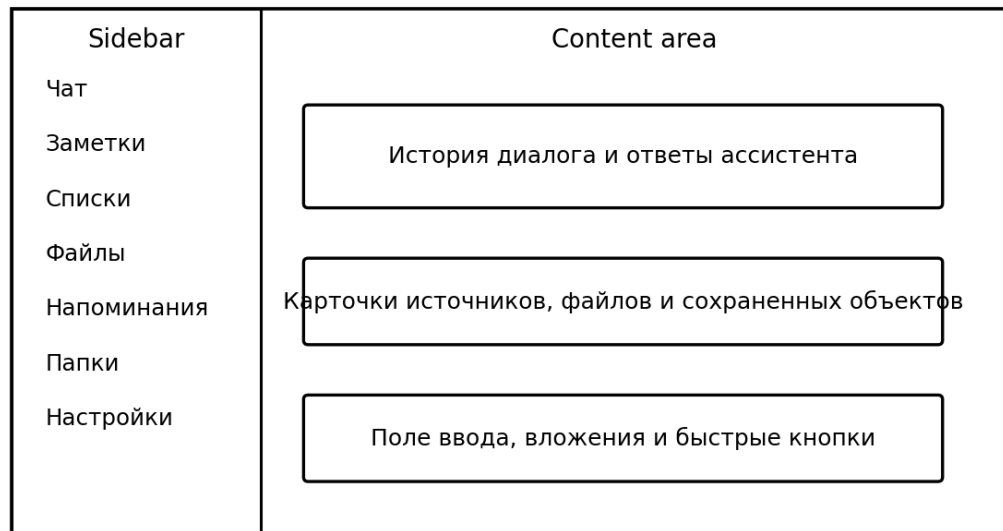
Тип файла	Способ обработки в MVP	Ограничения
DOCX	Извлечение текста через mammoth	Сложная верстка может передаваться упрощенно.
Изображения	Сохранение файла и метаданных	OCR в текущей версии не реализован.
PPTX	Сохранение файла и метаданных	Полный разбор презентаций вынесен в перспективы.
CSV	Сохранение и возможный текстовый просмотр	Нет отдельной аналитики табличных данных.

3.7 Проектирование интерфейса

При проектировании интерфейса использовался принцип: пользователь должен видеть знакомую структуру, но не должен сталкиваться с избыточными настройками. Поэтому основной экран напоминает чат, а дополнительные разделы вынесены в боковую навигацию. Это помогает объяснить идею проекта даже при первом запуске: в центр помещен ассистент, а рядом находятся сохраненные данные.

Особое внимание уделялось мобильной версии. На небольших экранах постоянное боковое меню занимает слишком много места, поэтому оно преобразуется в выдвижную панель. При этом поле ввода в чате остается доступным, а кнопки не должны перекрываться клавиатурой или нижней областью интерфейса.

Единый визуальный стиль важен не только с эстетической точки зрения. Если разделы заметок, файлов и напоминаний выглядят по-разному, пользователь воспринимает их как разные приложения. В MindVault все сущности отображаются через похожие карточки и действия, чтобы сохранить ощущение единой системы.



На мобильных экранах sidebar превращается в drawer, открываемый слева

Рисунок 7 - Схема пользовательского интерфейса MindVault

3.8 Выводы по главе

В ходе проектирования была определена клиент-серверная архитектура MindVault, описана модель данных и логика обработки пользовательских сообщений. Основой хранения стала универсальная сущность `item`, что позволило объединить файлы, заметки, списки и напоминания в одной пользовательской модели.

Проектирование также показало, что главная сложность системы заключается не в отдельных CRUD-операциях, а в правильной связи между чатом, ассистентом и реальными действиями backend. Именно это отличие делает MindVault не обычным заметочником, а персональным интеллектуальным хранилищем.

4 ТЕХНОЛОГИЧЕСКАЯ РЕАЛИЗАЦИЯ

4.1 Организация проекта

Практическая реализация MindVault выполнялась как разработка MVP, то есть минимально жизнеспособной версии продукта. В эту версию вошли основные функции, необходимые для демонстрации идеи: чат, файлы, заметки, списки, напоминания, папки, работа с ассистентом и адаптивный интерфейс. Такой подход позволил не расплываться на второстепенные возможности и довести до рабочего состояния ключевые сценарии.

Проект был организован как веб-приложение с разделением frontend и backend. Клиентская часть отвечает за пользовательский интерфейс и запросы к API. Серверная часть обрабатывает данные, работает с PostgreSQL, выполняет действия пользователя и обращается к внешнему поставщику ИИ при необходимости. Для запуска инфраструктуры используется Docker Compose, что упрощает подготовку демонстрационного стенда.

В ходе разработки применялась итерационная схема. Сначала были реализованы базовые разделы и хранение данных, затем добавлены папки и работа с файлами, после этого дорабатывалась логика ассистента и поведение интерфейса на нестандартных пользовательских вводах.

4.2 Реализация клиентской части

Клиентская часть реализована на React с использованием TypeScript. Компонентная модель React хорошо подошла для интерфейса MindVault, поскольку приложение состоит из повторно используемых элементов: карточек материалов, списка сообщений, формы ввода, панели навигации, кнопок действий и модальных окон.

Для сборки проекта использовался Vite. По сравнению с более тяжелыми сборочными решениями он обеспечивает быстрый запуск в режиме разработки и

удобную структуру для MVP. TypeScript использовался для описания типов данных, приходящих с сервера: item, folder, message, conversation. Это снизило вероятность ошибок при передаче данных между компонентами.

Визуальное оформление выполнено с помощью Tailwind CSS. Такой способ позволил быстро приводить интерфейс к единому стилю и точно исправлять адаптивность. Например, одинаковые отступы, скругления, размеры карточек и состояния кнопок задавались через общие классы, а не через разрозненные CSS-файлы.

Таблица 17 - Основные компоненты клиентского интерфейса

Компонент	Назначение	Особенности
ChatView	Отображение диалога	Показывает сообщения пользователя и ассистента.
MessageInput	Ввод сообщения и вложений	Поддерживает отправку текста и файлов.
Sidebar	Навигация	На мобильных экранах превращается в drawer-меню.
ItemCard	Карточка сохраненного материала	Используется для файлов, заметок, списков и напоминаний.
FolderList	Список папок	Позволяет фильтровать содержимое.
SettingsPanel	Настройки приложения	Содержит служебные параметры и пользовательские действия.

4.3 Реализация серверной части

Серверная часть реализована на Node.js и Express. Express был выбран как простой и понятный фреймворк для построения API. Он не навязывает сложную структуру проекта, что удобно для учебного MVP, но при этом позволяет явно разделять маршруты, обработчики и сервисную логику.

В серверной части были выделены несколько групп обработчиков: работа с папками, работа с items, обработка сообщений, загрузка файлов и взаимодействие с ассистентом. Запросы проходят проверку данных, после чего вызывается

соответствующая операция с базой данных. Для валидации использовалась Zod, что помогло избежать большого количества ручных проверок.

Особое внимание уделялось ошибкам. Для пользовательского приложения важно не только обработать успешный сценарий, но и корректно показать проблему: отсутствие текста, неподдерживаемый файл, ошибка API или базы данных. В таких случаях сервер должен возвращать понятный ответ, а клиент - отображать его без аварийного завершения интерфейса.

Таблица 18 - Слои серверной части

Слой	Реализация	Назначение
Routes	Express routes	Принимают HTTP-запросы от клиента.
Validation	Zod-схемы	Проверяют входные данные.
Services	Функции бизнес-логики	Создают items, folders и messages.
Storage	Drizzle ORM	Выполняет запросы к PostgreSQL.
AI adapter	Провайдер Gemini/API	Формирует запрос к языковой модели.
Error handling	Единая обработка ошибок	Возвращает понятные ответы клиенту.

4.4 Реализация базы данных

Схема базы данных была описана через Drizzle ORM. Выбор ORM упростил поддержку схемы, так как структура таблиц стала частью кода проекта. В отличие от ручного написания SQL в разных местах приложения, такой подход позволяет видеть модель данных централизованно и снижает риск несовпадения между кодом и реальной базой.

Для таблицы items поле type определяет смысл объекта. Это может быть note, file, list или reminder. Поле folder_id связывает материал с папкой. Поле metadata используется для дополнительных сведений, которые различаются в зависимости от типа объекта: размер файла, MIME-тип, дата напоминания, список пунктов, источники ответа ассистента.

Такой подход выбран как компромисс между строгостью реляционной модели и гибкостью персонального хранилища. Если для каждого вида объекта

создать отдельную таблицу, усложнится поиск и фильтрация по папкам. Если хранить все в одном JSON-файле, станет сложнее обеспечить целостность и нормальные запросы. Единая таблица items с типом объекта оказалась наиболее удобной для текущей версии.

4.5 Реализация загрузки и обработки файлов

Загрузка файлов в первой версии реализована с учетом демонстрационного характера проекта. Пользователь может прикрепить файл к сообщению или добавить его через раздел файлов. После этого сервер создает запись в базе и сохраняет метаданные. Если файл можно обработать как текстовый, система извлекает фрагменты содержимого для дальнейшей работы ассистента.

Для обработки DOCX использовалась библиотека mammoth, так как она позволяет получать текст из документа без необходимости запускать Microsoft Word. Для PDF применялся pdf-parse. При этом важно учитывать ограничение: если PDF является сканом, а не документом с текстовым слоем, извлечь содержимое без OCR невозможно. В текущей версии такие файлы сохраняются, но ассистент не заявляет, что прочитал их полностью.

В интерфейсе файл отображается как карточка с названием и дополнительной информацией. Для некоторых форматов доступен предпросмотр или текстовый фрагмент. Для неподдерживаемых форматов система должна сохранять файл и показывать, что интеллектуальный анализ ограничен.

4.6 Реализация заметок, списков и напоминаний

Заметки, списки и напоминания реализованы как типы item. Это позволило использовать одинаковые операции создания, получения, обновления и удаления. Различие между ними задается не отдельной логикой хранения, а типом и структурой содержимого.

Заметка содержит заголовок и текст. Список содержит набор пунктов, которые могут быть представлены в metadata или в структурированном содержимом. Напоминание содержит текст и дату выполнения. При создании напоминания система пытается разобрать простые временные выражения: «завтра», «сегодня», «послезавтра», «через час», «через 2 дня», «в 10:00».

Такой парсер не претендует на полное понимание всех вариантов русского языка. Его задача - стабильно обрабатывать наиболее частые сценарии, которые можно объяснить и протестировать. Более сложная обработка дат может быть добавлена в дальнейшем через специализированную библиотеку или отдельный модуль.

Таблица 19 - Типы сохраняемых объектов

Тип объекта	Основное содержимое	Пример команды
note	Текстовая заметка	заметка: идеи для раздела тестирования
list	Набор пунктов	список: купить бумагу, распечатать презентацию
reminder	Текст и дата	напомни завтра в 10 отправить работу
file	Файл и метаданные	прикрепление файла через интерфейс

4.7 Реализация ИИ-ассистента

ИИ-ассистент в MindVault используется как вспомогательный слой, а не как единственный механизм управления системой. Это принципиальное решение. Простые команды должны выполняться детерминированно, потому что их результат влияет на базу данных. Языковая модель подключается там, где требуется объяснение, суммаризация, помощь с текстом или классификация более сложного намерения.

В запрос ассистенту передается не вся база данных, а ограниченный релевантный контекст. Такой подход нужен по двум причинам. Во-первых, передача всей базы может быть избыточной и небезопасной. Во-вторых, языковая

модель имеет ограничение на размер контекста, поэтому важно выбирать только нужные фрагменты.

Ответ ассистента может содержать ссылки на источники, если он использовал сохраненные материалы. В текущей версии это реализуется через metadata сообщения. Такой механизм пока не заменяет подробные ссылки на конкретные фрагменты документов, но он повышает прозрачность: пользователь видит, что ответ связан с сохраненными объектами, а не полностью сгенерирован без контекста.

Таблица 20 - Разделение работы парсера команд и ИИ

Ситуация	Обработка	Причина
Явная команда создать заметку	Парсер команд, без ИИ	Действие должно быть надежным и проверяемым.
Обычный вопрос пользователя	ИИ, без создания объекта	Нужен содержательный ответ без сохранения лишних данных.
Просьба объяснить файл	Парсер + ИИ	Нужно использовать извлеченный текст и контекст.
Неоднозначная команда	Уточнение у пользователя	Система не должна создавать объект по сомнительному намерению.
Ошибка создания объекта	Сообщение об ошибке	Нельзя скрывать ошибку ответом модели.

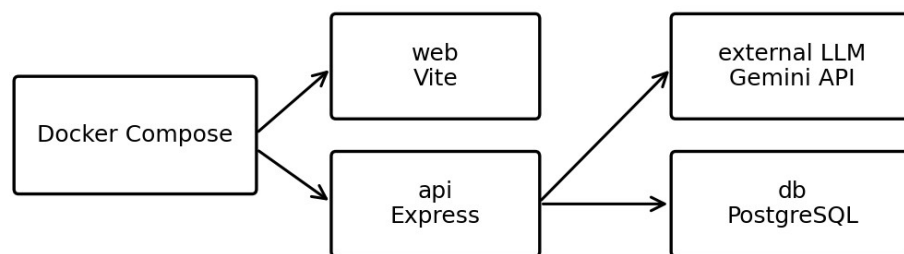
4.8 Адаптивность и доработка интерфейса

В ходе разработки отдельно проверялась мобильная версия. Первоначальные решения, удобные на desktop, не всегда хорошо работали на телефоне. Например, постоянное боковое меню занимало слишком много места, а кнопка прокрутки вниз могла перекрываться полем ввода. Поэтому для мобильного интерфейса были внесены отдельные доработки.

Боковое меню было приведено к логике drawer: кнопка открытия и направление появления панели должны быть согласованы. Поле ввода закреплено внизу так, чтобы не перекрывать область сообщений. Заголовки и элементы

интерфейса были приведены к единому шрифту, чтобы разные разделы не выглядели как части разных приложений.

Эти доработки важны для защиты проекта, потому что комиссия оценивает не только наличие функций, но и аккуратность реализации. Если интерфейс выглядит небрежно, у проверяющего может возникнуть ощущение, что проект собран на скорую руку, даже если техническая часть работает корректно.



Сценарий демонстрации: `docker compose up -d --build`, затем проверка web и api

Рисунок 8 - Схема развертывания MindVault

4.9 Обработка ошибок и устойчивость

Для пользовательского приложения важно, чтобы некорректный ввод не приводил к аварийному состоянию. Поэтому в MindVault были предусмотрены проверки пустого сообщения, команды без содержания, неподдерживаемого файла, ошибки ответа ИИ и ошибки сохранения в базе данных.

Если ассистент недоступен, приложение не должно терять уже сохраненные материалы. В таком случае пользователь получает сообщение об ошибке внешнего сервиса, но локальные разделы файлов, заметок и папок продолжают работать. Это особенно важно для демонстрации: отсутствие ответа внешнего API не должно полностью останавливать приложение.

Отдельно проверялась ситуация, когда пользователь вводит странную фразу из нескольких ключевых слов. Система не должна пытаться угадать намерение

слишком уверенно. Более безопасное поведение - запросить уточнение или дать обычный ответ без создания лишних объектов.

4.10 Выводы по главе

В результате реализации был создан рабочий прототип MindVault, включающий клиентскую и серверную части, базу данных, работу с файлами, заметками, списками, напоминаниями, папками и ИИ-ассистентом. Архитектура проекта допускает дальнейшее расширение без полного переписывания системы.

Основной технический результат состоит в том, что чатовый интерфейс был связан с реальными backend-операциями. Пользовательские команды не просто интерпретируются текстом ассистента, а приводят к созданию объектов в базе данных при выполнении условий.

5 ТЕСТИРОВАНИЕ И АНАЛИЗ РАБОТЫ СИСТЕМЫ

5.1 Методика тестирования

Тестирование MindVault проводилось по сценариям, связанным с основной задачей приложения: сохранением и обработкой пользовательской информации. Проверялись как стандартные действия, так и случаи, которые могут привести к ошибкам: пустые сообщения, неоднозначные команды, неподдерживаемые файлы, недоступность ИИ-сервиса.

Так как проект является MVP, тестирование было направлено не на промышленную нагрузку с большим числом пользователей, а на проверку полноты и устойчивости пользовательских сценариев. Это соответствует этапу разработки выпускной квалификационной работы, где важно показать работоспособность подсистем, их интеграцию и понимание ограничений.

Для каждого сценария фиксировались входные данные, ожидаемый результат, фактическое поведение и статус. Такой подход позволяет не только показать успешные функции, но и честно отделить реализованные возможности от перспектив развития.

Таблица 21 - Параметры тестирования

Параметр	Значение
Тип тестирования	Функциональное, интерфейсное, проверка нестандартных вводов.
Платформа	Веб-браузер на desktop и mobile-разрешениях.
База данных	PostgreSQL в окружении проекта.
Проверяемые сущности	folders, items, conversations, messages.
Проверяемые типы items	note, list, reminder, file.
Особое внимание	Надежность действий ассистента и отсутствие ложных сохранений.

5.2 Функциональное тестирование

Функциональное тестирование показало, что основные сценарии приложения выполняются корректно. Пользователь может создать заметку, список, напоминание, загрузить файл, создать папку и отфильтровать материалы по выбранной папке. При успешном действии интерфейс получает данные с сервера и отображает обновленное состояние.

Особенно важно было проверить, что обычные вопросы к ассистенту не сохраняются как заметки. Такая ошибка была выявлена при раннем тестировании логики приложения и могла полностью испортить логику приложения: вместо персонального хранилища пользователь получил бы базу нерелевантных записей из всех сообщений чата.

Результаты тестирования представлены в таблице. В таблицу включены не только идеальные сценарии, но и реальные ситуации, которые могли возникнуть при демонстрации приложения.

Таблица 22 - Результаты функционального тестирования

№	Сценарий	Результат проверки	Статус
1	Создание заметки через команду	Появляется item типа note, заметка создана и отображается.	пройдено
2	Создание списка	Пункты сохраняются как список.	пройдено
3	Создание напоминания 'завтра в 10'	Дата рассчитана и сохранена.	пройдено
4	Загрузка файла	Файл появляется в разделе файлов и сохраняется с метаданными.	пройдено
5	Создание папки	Папка доступна в навигации.	пройдено
6	Фильтрация по папке	Показываются связанные материалы.	пройдено
7	Обычный вопрос ассистенту	Ответ формируется без создания item.	пройдено
8	Удаление объекта	Объект исчезает из списка.	пройдено

5.3 Тестирование нестандартных пользовательских вводов

Нестандартные вводы являются важной частью тестирования именно для чатового приложения. В форме пользователь ограничен полями, а в чате он может

написать что угодно. Поэтому система должна быть устойчивой не только к правильным командам, но и к неточным формулировкам.

В ходе тестирования проверялись пустые сообщения, команды без содержимого, смешанные ключевые слова, длинные тексты и обычные просьбы ассистенту. Главный критерий: система не должна создавать данные без достаточного основания и не должна аварийно завершаться при непонятном вводе.

Таблица 23 - Тестирование нестандартных пользовательских вводов

Ввод	Ожидаемое поведение	Результат
пустая строка	Сообщение не отправляется	Ошибка не возникает.
заметка	Запросить текст заметки	Объект не создается.
список	Запросить пункты списка	Объект не создается.
напоминание	Запросить текст и время	Объект не создается.
заметка список напоминание	Не создавать несколько объектов	Система требует уточнения.
что ты умеешь?	Ответить как ассистент	Item не создается.
большой текст без команды	Обработать как сообщение	Система не зависает.
создай что-нибудь	Запросить уточнение	Лишние данные не создаются.

5.4 Тестирование работы с файлами

Проверка файловой подсистемы включала загрузку файлов разных форматов и оценку того, как система ведет себя при успешном и ограниченном извлечении содержимого. Для MVP важно было, чтобы файл сохранялся даже тогда, когда его невозможно полноценно прочитать.

По итогам проверки текстовые форматы и DOCX подходят для дальнейшей обработки лучше всего. PDF обрабатывается при наличии текстового слоя. Изображения и презентации пока сохраняются как файлы с метаданными. Это ограничение отражено в работе как направление дальнейшего развития, а не как реализованная функция.

Таблица 24 - Проверка работы с файлами разных форматов

Формат	Проверяемое действие	Результат	Статус
TXT	Загрузка и чтение текста	Текст доступен для обработки	пройдено
DOCX	Извлечение текста	Содержимое извлекается в упрощенном виде	пройдено
PDF	Извлечение текста из документа	Работает для PDF с текстовым слоем	частично
PNG/JPG	Сохранение файла	Файл сохраняется, OCR отсутствует	частично
PPTX	Сохранение файла	Файл сохраняется, полный разбор не реализован	частично
Неизвестный тип	Безопасная обработка	Система не завершается аварийно	пройдено

5.5 Тестирование адаптивности интерфейса

Адаптивность проверялась на типовых ширинах экрана. Основной задачей было убедиться, что чат остается пригодным для использования, а навигация не перекрывает содержимое. На desktop-экране удобно использовать постоянное боковое меню. На мобильных экранах оно должно скрываться и открываться по кнопке.

Проверка показала, что адаптивная логика позволяет демонстрировать проект как на ноутбуке, так и на телефоне. Это важно, потому что персональное хранилище относится к приложениям, которые пользователь может открывать не только за рабочим столом, но и в момент быстрых заметок.

Таблица 25 - Проверка адаптивности интерфейса

Разрешение	Проверка	Результат
Desktop 1440 px	Боковое меню и чат	Элементы отображаются корректно.
Laptop 1280 px	Карточки и навигация	Интерфейс не ломается.
Tablet 768 px	Переход к компактному виду	Навигация остается доступной.
Mobile 390 px	Drawer-меню и поле ввода	Основной сценарий выполняется.

Разрешение	Проверка	Результат
Mobile 360 px	Кнопки и нижняя область	Поле ввода не перекрывает критичные элементы.

5.6 Анализ эффективности решения

Эффективность MindVault проявляется не в абсолютной скорости вычислений, а в сокращении числа переходов между разными инструментами. Для пользователя важнее не то, за сколько миллисекунд создается заметка, а то, что ему не нужно открывать отдельный заметочник, облачный диск и планировщик задач.

По сравнению с ручным способом хранения информации MindVault дает более связанный сценарий. Пользователь может загрузить материал, обсудить его с ассистентом и сохранить результат в той же системе. Даже если часть возможностей пока реализована как MVP, архитектура уже поддерживает дальнейшее развитие в сторону полноценного персонального цифрового пространства.

Таблица 26 - Сравнение MindVault с разрозненным способом хранения

Критерий	Разрозненные сервисы	MindVault
Сохранение файла	Облачный диск	Раздел файлов и чат в одной системе.
Быстрая заметка	Заметочник или мессенджер	Команда в чате или отдельный раздел заметок.
Напоминание	Календарь или task-менеджер	Команда в чате и карточка напоминания.
Работа с контекстом	Нужно вручную копировать данные	Ассистент получает релевантные сохраненные материалы.
Сортировка	Отдельные папки в разных сервисах	Папки внутри единой модели данных.
Простота демонстрации	Нужно показывать несколько приложений	Один веб-интерфейс.

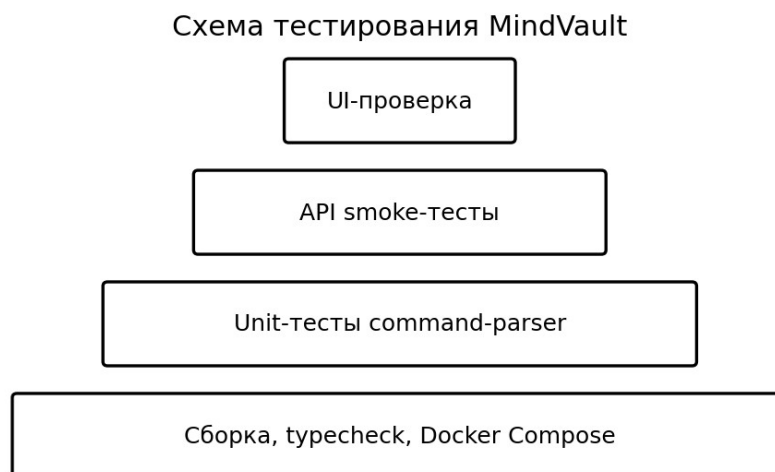


Рисунок 9 - Схема проверенных сценариев тестирования

5.7 Ограничения текущей версии

Текущая версия MindVault является рабочим прототипом, но не промышленной системой. Это важно явно зафиксировать, чтобы не завышать результаты работы. В приложении уже реализованы основные пользовательские сценарии, однако некоторые функции требуют дальнейшего развития.

К ограничениям относятся отсутствие полноценного семантического поиска, отсутствие векторной базы данных, ограниченная обработка изображений и презентаций, отсутствие совместной работы нескольких пользователей и недостаточно развитые настройки приватности. Кроме того, для промышленного внедрения необходимо настроить домен, HTTPS, резервное копирование, более строгую авторизацию и мониторинг ошибок.

Эти ограничения не отменяют результата ВКР, поскольку цель работы состояла в разработке прототипа и проверке архитектурного подхода. Наоборот, наличие понятного списка ограничений показывает, что проект может развиваться планомерно, а не ограничиваться демонстрацией отдельных экранов.

Таблица 27 - Ограничения текущей версии и план развития

Ограничение	Причина	План развития
Нет полноценного	Требуется векторный индекс	Подключить embeddings и векторную БД.

Ограничение	Причина	План развития
семантического поиска		
Нет OCR для изображений	Нужна отдельная OCR-подсистема	Добавить распознавание текста на изображениях.
Ограниченный разбор PDF	Сканированные PDF не имеют текстового слоя	Использовать OCR или внешний сервис обработки документов.
Нет совместной работы	MVP ориентирован на одного пользователя	Добавить роли, доступы и совместные папки.
Нет мобильного приложения	Реализован только адаптивный web	Создать PWA или мобильный клиент.
Ограниченная production-безопасность	Учебный прототип	Настроить HTTPS, резервное копирование и мониторинг.

5.8 Выводы по главе

Тестирование подтвердило, что MindVault выполняет основные сценарии: создание заметок, списков и напоминаний, загрузку файлов, работу с папками, ответы ассистента и корректную обработку нестандартных вводов. Особое значение имеет проверка того, что обычные вопросы не сохраняются автоматически как пользовательские материалы.

Система показала применимость выбранной архитектуры для задачи персонального интеллектуального хранилища. При этом текущая версия честно рассматривается как MVP, который может быть расширен семантическим поиском, лучшей обработкой документов, совместной работой и production-настройками.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы было разработано веб-приложение MindVault, предназначенное для хранения и интеллектуальной обработки пользовательских данных. Работа началась с анализа проблемы разрозненного хранения информации и существующих аналогов, после чего были сформулированы требования к системе и определена ее архитектура.

В результате разработки создан прототип, объединяющий чатовый интерфейс, файлы, заметки, списки, напоминания и папки. Отдельное внимание уделено логике ассистента. В системе разделены обычные ответы ИИ и реальные действия приложения: заметка, список или напоминание создаются только при наличии понятной команды и успешном выполнении операции на сервере.

В ходе проектирования была выбрана клиент-серверная архитектура. Клиентская часть реализована с использованием React, Vite, TypeScript и Tailwind CSS. Серверная часть построена на Node.js и Express. Для хранения данных используется PostgreSQL, работа со схемой организована через Drizzle ORM, а валидация входных данных - через Zod.

Проведенное тестирование показало, что основные пользовательские сценарии выполняются корректно. Система устойчиво обрабатывает пустые сообщения, команды без содержания, смешанные ключевые слова, загрузку файлов разных форматов и обычные вопросы к ассистенту. Это подтверждает достижение цели работы и применимость разработанного подхода.

Практическая значимость MindVault состоит в том, что приложение может использоваться как основа персонального цифрового пространства. Пользователь получает не отдельный файловый менеджер или заметочник, а связанную систему, где материалы можно сохранять, сортировать и обсуждать с ассистентом в одном интерфейсе.

Перспективы развития проекта включают подключение семантического поиска, использование векторной базы данных, улучшенное извлечение текста из PDF, DOCX, изображений и презентаций, календарную интеграцию, совместную работу пользователей, расширенные настройки приватности, мобильное приложение или PWA, а также полноценное production-развертывание с доменом и HTTPS.

Таким образом, поставленные задачи выполнены: проведен анализ предметной области, сформулированы требования, спроектирована архитектура, реализованы основные подсистемы и проведено тестирование. Разработанное приложение демонстрирует возможность объединения персонального хранилища и интеллектуального ассистента в едином веб-интерфейсе.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 7.32–2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления.
2. ГОСТ Р 7.0.5–2008. Библиографическая ссылка. Общие требования и правила составления.
3. Басова А. В., Боженко С. В., Вахнина Т. Н. и др. Правила оформления текстовых документов. Кострома: КГУ, 2017.
4. Красавина М. С., Барило И. И. Выпускная квалификационная работа: порядок выполнения и защиты. Кострома: КГУ, 2018.
5. Sommerville I. Software Engineering. 10th ed. Pearson, 2016.
6. Pressman R. S., Maxim B. R. Software Engineering: A Practitioner's Approach. McGraw-Hill, 2020.
7. Fowler M. Patterns of Enterprise Application Architecture. Addison-Wesley, 2002.
8. Gamma E., Helm R., Johnson R., Vlissides J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
9. Nielsen J. Usability Engineering. Morgan Kaufmann, 1993.
10. Norman D. The Design of Everyday Things. Basic Books, 2013.
11. Krug S. Don't Make Me Think. New Riders, 2014.
12. React Documentation. URL: <https://react.dev/> (дата обращения: 20.05.2026).
13. Vite Documentation. URL: <https://vite.dev/> (дата обращения: 20.05.2026).
14. TypeScript Documentation. URL: <https://www.typescriptlang.org/docs/> (дата обращения: 20.05.2026).
15. Tailwind CSS Documentation. URL: <https://tailwindcss.com/docs> (дата обращения: 20.05.2026).
16. Node.js Documentation. URL: <https://nodejs.org/docs/latest/api/> (дата обращения: 20.05.2026).

17. Express Documentation. URL: <https://expressjs.com/> (дата обращения: 20.05.2026).
18. PostgreSQL Documentation. URL: <https://www.postgresql.org/docs/> (дата обращения: 20.05.2026).
19. Drizzle ORM Documentation. URL: <https://orm.drizzle.team/docs/overview> (дата обращения: 20.05.2026).
20. Zod Documentation. URL: <https://zod.dev/> (дата обращения: 20.05.2026).
21. TanStack Query Documentation. URL: <https://tanstack.com/query/latest> (дата обращения: 20.05.2026).
22. Docker Documentation. URL: <https://docs.docker.com/> (дата обращения: 20.05.2026).
23. OWASP Foundation. OWASP Top Ten Web Application Security Risks. URL: <https://owasp.org/www-project-top-ten/> (дата обращения: 20.05.2026).
24. Google AI for Developers. Gemini API Documentation. URL: <https://ai.google.dev/gemini-api/docs> (дата обращения: 20.05.2026).
25. OpenAI. Prompt engineering and safety materials. URL: <https://platform.openai.com/docs/> (дата обращения: 20.05.2026).
26. pdf-parse package documentation. URL: <https://www.npmjs.com/package/pdf-parse> (дата обращения: 20.05.2026).
27. mammoth package documentation. URL: <https://www.npmjs.com/package/mammoth> (дата обращения: 20.05.2026).